Oscar Esparza
Jose L. Muñoz
Juanjo Alins
Jorge Mata

UPC Network Engineering Department

# SIP

# Contents

**1 SIP** **5**
1.1 Practices . . . . . 5
1.1.1 Basics of SIP . . . . . 5
1.1.2 SIP with Proxies . . . . . 13

# Chapter 1

# SIP

## 1.1 Practices

### 1.1.1 Basics of SIP

#### Scenario

The objective of this lab session is to put you in touch with the basic concepts of SIP signaling. The scenario you are going to use is depicted in Figure 1.1. Prior to start with the session, it is strongly recommended to have some knowledge about the DNS (Domain Name System), SDP (Session Description Protocol), RTP (Real Time Protocol) and RTCP (Real Time Control Protocol).

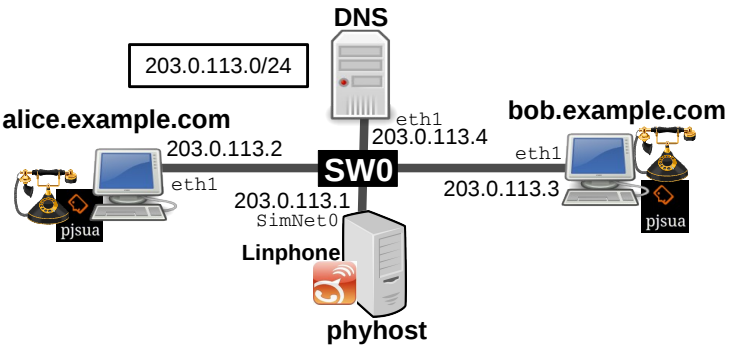



Figure 1.1: Scenario for the SIP lab session

To start the scenario execute in the HOST machine:

```
HOST$ simctl sip-basic start
```

#### Direct call between *pjsua* clients

**Exercise 1.1–** In this exercise you are going to make a direct call (no intermediaries) between **alice** and **bob** using the application *pjsua*. You will capture the traffic using `wireshark` to analyze the basics of the SIP signaling.

Notice that the calling user **alice** has to know, at least, the IP address (or DNS name) of the callee user **bob**. By default, *pjsua* sends SIP packets using UDP as transport protocol and port number 5060, but as you will see, this port number can be changed. RTP and RTCP also use UDP, but the port numbers are generally announced in the session establishment.

- Execute in **alice** the following command:

```
alice:~# pjsua --null-audio --local-port=5060
```

  This command opens the SIP client (calling party). In this particular case the option *--local-port* is unnecessary as you are going to use the default port, but now you know how to change the value of this port.

- Into the subshell of *pjsua* in **alice**, set the codec priority (**Cp**) to the value 200 for the PCMA codec. This will make this codec to be the first option to choose.

```
>>>Cp
Codec name ("*" for all) and priority: PCMA/8000 200
```

  Most of the calls you will make are between virtual machines. Our virtual machines have no input/output devices for audio (there are no headphones or speakers in the virtual machines). So, you will use `wireshark` to "listen" to these conversations. The problem is that *wireshark* does not properly decodify the default codec speex, and for this reason we transmit using the PCMA codec, which is properly decodified by `wireshark`.

- Start *pjsua* in **bob** (callee party) with the following options:

```
bob:~# pjsua --play-file /root/hello16000.wav --auto-answer 180 --auto-play \
--local-port=5000 --null-audio
```

  This command configure *pjsua* to answer automatically (`-auto-answer` option) with code 180 (which corresponds to "Ringing"), and to play directly the audio file `/root/hello16000.wav` when accepting the call. Also, in **bob** you are going to listen to local port 5000, not by the default SIP port (5060).

- Open wireshark to listen to **SimNet0**.

- Call **bob** from **alice** using option `m` and introducing the URI of **bob**:

```
>>> m
(You currently have 0 calls)
Buddy list:
 -none-

Choices:
   0          For current dialog.
  -1          All 0 buddies in buddy list
  [1 - 0]     Select from buddy list
  URL         An URL
  <Enter>     Empty input (or 'q') to cancel
Make call: sip:bob@bob.example.com:5000
```

  Notice that instead of the DNS name, you can also use bob's IP address (sip:bob@203.0.113.3:5000).

- Accept the call in *pjsua* of **bob** by typing **"a"**, and answering with a `200` code (`OK`):

```
 Press a to answer or h to reject call
a
Answer with code (100-699) (empty to cancel): 200
```

- Wait for 10 seconds approximately, and finish the call by typing **"h"** in any of the two virtual machines. As we mentioned previously, you did not listen anything because virtual machines do not have any input/output device. Later you will use `wireshark` and the traffic capture to listen it.

- Stop the capture of `wireshark` and let us analyze the traces:

  1. How many packets there are in the traces?

2. Which protocols can you see?

As you can see, there can be hundreds of packets, depending on the time you lasted to finish the call. You will see DNS, SIP, RTP and RTCP packets. Stop the capture and save it in a file, so you can later study packets later.

- First, have a look to the DNS packets (to see only DNS packets, put `dns` in the Filter box in the left-upper part of wireshark, and press "Apply")

  1. How many DNS packets can you see?
  2. Who are the origin and destination of these packets?
  3. What is the purpose of these messages?

  You will probably see only two DNS packets, a query and a response. The query is made by alice to the DNS server to ask which is the address that corresponds to the name `bob.example.com`. The DNS sends in the response an A record, that contains the corresponding mapping. Notice that this is the most simplistic exchange of information that you can perform with a DNS server. Later in this lab session, you will have to analyze more complicated dialogues with the DNS, because there are many features of SIP that use the DNS. So, if you do not remember the concepts related to DNS, you had better to review them because we will assume that you know them.

- Now analyze `sip` packets (filter them using wireshark, as you did with `dns` packets).

  1. How many SIP packets can you see?
  2. Who are the origin and destination of these packets?
  3. Which of these messages contain SDP information?
  4. Describe de dialogue you see.

- You are now going to see some features that wireshark offers to analyze VoIP calls. Go to menu: $Telephony \rightarrow VoIPcalls$

  1. How many calls can you see in this view?
  2. Which information can you obtain from this summary? Try to identify what is the meaning of each field.

  As you can see, this view is just a snapshot of the VoIP calls captured by `wireshark`. In this case, you can see the summary of this only call. The fields represented are:

  - **Start Time**: Relative time of the start of the call regarding the start of the capture.
  - **Stop Time**: Relative time of the end of the call regarding the start of the capture.
  - **Initial Speaker**: IP address of the calling party.
  - **From**: This corresponds to the information contained in the field “From” of the INVITE.
  - **To**: This corresponds to the information contained in the field “To” of the INVITE.
  - **Packets**: Number of packets exchanged during the call.
  - **State**: Status of the call. Possible values are:
    - **CALL SETUP**: Setup, Proceeding, Progress or Alerting
    - **RINGING**: The callee sent the ring tone
    - **IN CAL**L: Call established
    - **CANCELLED**: Calling party cancelled before establishment.
    - **COMPLETED**: Call established and ended.
    - **REJECTED**: Callee canceled before establishment.
    - **UNKNOWN**: Unknown status of the call.

- Another typical feature of wireshark is showing the dialogue of a call in a graphical way, so go to:

  $Telephony \rightarrow VoiPcalls \rightarrow Selectall \rightarrow Flow$

  1. Can you identify the addresses that exchange SIP packets?
  2. Identify the port numbers that are used for each entity and protocol.

  As you can see, this view shows the time graph of the call. In this particular case, there are only two participants, but there can be up to 10 columns, each one representing the IP address of one of the principals. Each conversation uses a different color, and the arrows show the direction of each packet. Another particular feature of this view is that the RTP traffic is summarized in a bigger arrow.

- Anyway, if you want to see the complete SIP dialogue, with much more information about packets and headers, there is another view. Click on the packet that corresponds to the INVITE method and select:

  $Analyze \rightarrow FollowUDPStream$

  This view shows to you all the SIP dialogue. Save this dialogue in ASCII mode in a file (for instance in the Desktop folder), and open it.

- Let us first have a look to the INVITE message:

  1. Is the INVITE message a Request or a Response?
  2. Is this INVITE message following the format described in the theoretical lectures (Start line+Headers+Empty line+Body).
  3. In the start line of the INVITE (also known as `request line`), what do you see? Can you identify the method of this message? And the SIP URI? And the version of the protocol?

  As you can see, INVITE is the first request message sent by the SIP client, and the `start line` contains a lot of relevant information. Notice that INVITE is the method, and the SIP URI is responsible for the routing of this request.

  Now let us analyze some of the headers:

  1. Have a look to the `Via` header, as there are some important fields. Can you identify them?
  2. What is the transport protocol used for SIP in this particular case?
  3. What do you think the `branch` field is? Is this `branch` value repeated in other messages? (You can make a search of this value in the text file).

  INVITE is the first message of the first **transaction** to open a SIP session (a transaction is a request/response conversation between two adjacent SIP entities), and in our particular case is formed by INVITE, Trying, Ringing and 200 OK. The `branch` field contains the identifier of the transaction. Try to locate other transactions in this call, as they will have a different `branch` value.

- Have a look now to the `From`, `To` and `Call-ID` headers:

  1. What information is contained in these headers?
  2. Are the values of `From` and `To` changing in the following SIP messages?
  3. And the `tag` field?
  4. Is the value of the `Call-ID` header repeated in other SIP messages?

  Notice that `To` and `From` contain information related to who the sender and receiver of this request are. However, they are not used to route the request (requests are routed using the `SIP URI` of the `start line`). In the request, they are used to fully identify the **dialogue**. All the messages involved in a call form a **dialogue**, which is identified unambiguously by `To`, `From` and `Call-ID`.

- There are many other headers available in the INVITE message, but now we are not going to describe them. Have a look to the `Message Body`.

    1. Can you see a SDP description in this `Message Body`? Which are the media proposed by the client (`m=`) in the INVITE request?

    In the message body a SDP profile is transmitted to inform the server about the preferred media to be used. In this particular case, the client propose a list of audio codecs:

```
m=audio 4000 RTP/AVP 8 98 97 104 3 0 99 9 96
```

    And later a clarification about which are particularly these codecs:

```
a=rtpmap:8 PCMA/8000
a=rtpmap:98 speex/16000
a=rtpmap:97 speex/8000
a=rtpmap:104 iLBC/8000
a=fmtp:104 mode=30
a=rtpmap:3 GSM/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:99 speex/32000
a=rtpmap:9 G722/8000
a=rtpmap:96 telephone-event/8000
a=fmtp:96 0-16
```

    As you can see, the PCMA/8000 codec is the preferred one (remember you changed this configuration in a previous step in `pjsua`).

- Have a look to the messages following the INVITE. Have a look to the `Trying` message:

    1. Is `Trying` a request or a response message?
    2. Which is the code for the Trying messages? According to this code, which kind of response is this message (Provisional, Success, Redirection, etc.)
    3. Do you know any other application layer protocol that use these text codes for messages, for instance `[404 - Not Found]`?
    4. Is `Trying` part of the same transaction than INVITE? How do you know that?

    `Trying` is a provisional response that indicates that the request has been received by the next-hop and some time is necessary to perform an action (like consulting a database). Provisional Responses may appear or not in a dialogue. As you can see, the first line of response messages include the SIP version and a code. This syntax is similar to that of HTTP.

- Now, have a look to the `Ringing` message:

    1. Request or response? Type of response? Which code? Same transaction?
    2. Which are the differences with the previous message `Trying`?
    3. In the `Allow` header, what can you see?

    The `Ringing` message is another provisional response that is usually sent when the destination UA receives the INVITE and it is trying to alert the user. The `Allow` header is used by one UA to inform the other UA about the methods (requests) that is able to understand.

- Have a look to the following message `OK`:

    1. Request or response? Type of response? Which code? Same transaction?
    2. Are the methods supported by **alice** the same than the ones of **bob**?

3. What can you see in the message body of `200 OK`?
4. What is finally the media selected for the transmission of the audio?

As you can see `200 OK` is the success response, meaning that the INVITE request was successful. After these messages, all the session parameters have been informed to both participants. You can see SDP information inside the message body, stating that finally the `8 PCMA/8000` codec is selected for the audio.

- Have a look to the `ACK` message:

1. Request or response? Same transaction?

The ACK is an atomic message, and it is not considered nor a request nor a response. Depending on if the previous response message is successful or not, `ACK` could be part of the starting transaction or not. In this particular case in which the request-response exchange has been successful, the `ACK` message is within a different transaction (you can see that it has a different `branch`), so the starting of a successful session using SIP comprises two transactions (INVITE-OK and ACK). This INVITE-OK-ACK exchange of messages is also called SIP 3-way handshake[1].

- After this SIP 3-way handshake there are other kind of packets, and you will not see them in the file you exported with the SIP dialogue. So return again to the `wireshark` traces:

1. Apart from DNS, SIP and SDP, which other protocols can you see in wireshark?
2. What is the purpose of these protocols?
3. What transport protocol do they use? Which are the ports for sender and receiver?
4. Try to find these port numbers in the SIP 3-way handshake. Do you find them? Where?

You will see in wireshark lots of RTP packets, and some RTCP packets. RTP is intended to transmit the audio, and RTCP to provide some feedback about this audio transmission. RTP and RTCP use UDP as transport protocol, and the ports numbers are informed in the SIP 3-way handshake, using the SDP information of `INVITE` and `OK` messages.

- Now have a look to the two ending SIP messages:

1. Which are the messages? Which one is a request, and which is a response?
2. Are these messages in the same transaction?
3. What is the purpose of these messages? Who finished the call? Do you think that this is the only way of closing a call?

   As you can see, these are the messages used to close a session, and it is enclosed in a single transaction. These are the ending messages to finish the dialogue. Remember which were the fields to identify the transaction (the call) in the SIP headers, and verify that all the previous messages have (almost) the same values in these fields.

- Finally, we are going to listen to the call. Go the the wireshark menu: $Telephony \rightarrow VoIP \rightarrow Player \rightarrow Decode \rightarrow Play$
  This will allow you to listen to the PCMA flow.

The objective of this first exercise was to get in touch with SIP signaling, so you can know which are the typical packets, fields and values involved to create, maintain and finish a session. In the following exercises you will make some few different calls, and to fully understand them it is necessary you dominate all this previous concepts.

**Exercise 1.2–**

Now you will test some unsuccessful calls.

[1]Do not confuse this SIP 3-way handshake with the TCP 3-way handshake (SYN-SYN/ACK-ACK). SIP is an application layer protocol, unlike TCP, which is a transport level protocol. Notice that SIP may or may not use TCP as transport protocol.

- Call **bob** from **alice** using option `m`, but now when introducing the URI of **bob** do not include the port number:

```
>>> m
(You currently have 0 calls)
Buddy list:
 -none-

Choices:
   0         For current dialog.
  -1         All 0 buddies in buddy list
  [1 - 0]    Select from buddy list
  URL        An URL
  <Enter>    Empty input (or 'q') to cancel
Make call: sip:bob@bob.example.com
```

- Wait some seconds, and close the call by typing "h".

  1. What SIP messages can you see in wireshark?
  2. What other messages can you see in wireshark?
  3. What do you think it is happening?

- Now include again the port number in the call:

```
>>> m
(You currently have 0 calls)
Buddy list:
 -none-

Choices:
   0         For current dialog.
  -1         All 0 buddies in buddy list
  [1 - 0]    Select from buddy list
  URL        An URL
  <Enter>    Empty input (or 'q') to cancel
Make call: sip:bob@bob.example.com:5000
```

- Do nothing in **bob**, just wait a couple of minutes looking at wireshark traces. After this time, type "h" in **alice**.

  1. Can you see any new messages? Describe briefly the dialogue.
  2. What do you think is happening? Think about a telephone call.
  3. What is the purpose of a `CANCEL` message? Request or response? Same transaction?
  4. Who is sending the `487 Request terminated` message? Request or response?

- Now do the same previous call, but this time type "h" in the console of **bob**.

  1. Can you see any new SIP message? What is the code of that message?
  2. What do you think is happening? Think about a telephone call.

There are many other ways of closing a session, and many other dialogues possible, these were just some examples. Feel free to test other kind of calls.

### Call with Softphones

**Exercise 1.3–**

In this exercise we are going to make a call using a softphone, so you can hear the audio. You will see that there are no differences regarding SIP signaling between `pjsua` and `linphone`.

- Configure the virtual interface `SimNet0` in the HOST machine with this command:

```
HOST$ ifconfig SimNet0 203.0.113.1/24
```

- Go the start menu of the HOST machine and open `linphone`.
- Start `wireshark` in `SimNet0`.
- Make a call from `linphone` in HOST to `pjsua` in **bob**. Use the IP address (and the proper port number) instead of the DNS name of **bob**.

  1. Which is the default port of the `linphone` client? Execute the following command to see this:

```
HOST$ netstat -nlp |grep linphone
```

  2. Can you tell any difference regarding to the previous calls?

  Every softphone has its own characteristics. You can have a look to the configuration parameters of linphone to have an idea.

### 1.1.2 SIP with Proxies

#### Scenario

The objective of the following lab exercises is to introduce concepts related to SIP proxies. First of all, stop the previous scenario and start the new one:

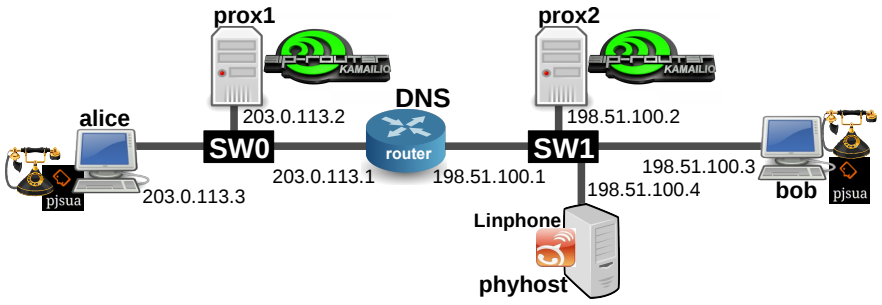



Figure 1.2: Scenario of SIP with proxies

```
HOST$ simctl sip-basic stop
HOST$ simctl sip-2proxies start
```

This new scenario is depicted in Figure 1.2, and it has the shape of a trapezoid, and clients will use proxies to help them to make calls. These proxies are implemented using the `kamailio` server.

#### Configuration of the Kamailio database

Before making calls, we have to configure the kamailio database.

- First, we have to create the database that will store all the information related to authorized users. To do so, execute the following commands in both proxies **prox1** and **prox2**:

  ```
  prox1# kamdbctl create
  prox2# kamdbctl create
  ```

  You will be asked for the password of the root user of mysql. This password does not exists, so just press "Enter". To finish creating the database, just answer "yes" to all questions in the console.

- Include the SIP users in the corresponding proxy, so they can register and make/receive calls:

  ```
  prox1# kamctl add alice xxxx
  ```

  ```
  prox2# kamctl add bob xxxx
  ```

  You will be asked for the password of the user with write permission in the kamailio database. This password is **kamailiorw**.

- You can see the information related to these new users:

  ```
  proxX$ kamctl db show subscriber
  ```

Notice that you did not state the domain of the users. However in the properties you can see that there is a by default domain. *prox1* has been configured to provide service in the domain **example.org** and *prox2* in the domain **example.com**.

- Finally, you can start the kamailio servers:

```
prox1$ service kamailio start
prox2$ service kamailio start
```

We recommend doing this configuration manually (at least once), but you can also use a simctl label `kamdb` to do so:

```
HOST$ simctl sip-2proxies exec kamdb
```

### Configuration of name resolution in wireshark

- You are going to configure the name resolution in `wireshark`, so you can better identify the hosts that exchange packets. Introduce this contents in the file `$HOME/.wireshark/hosts` (if the file does not exist, create it):

```
203.0.113.1     dnsA
203.0.113.2     prox1
203.0.113.3     alice
198.51.100.1    dnsB
198.51.100.2    prox2
198.51.100.3    bob
198.51.100.4    linphone
```

- Next, open `wireshark` and go to the menu Edit->Preferences->Name Resolution, and tag *Only use the profile "hosts" file* and *Resolve network (IP) addresses* (see Figure 1.3). Save changes an close `wireshark` for later usage.

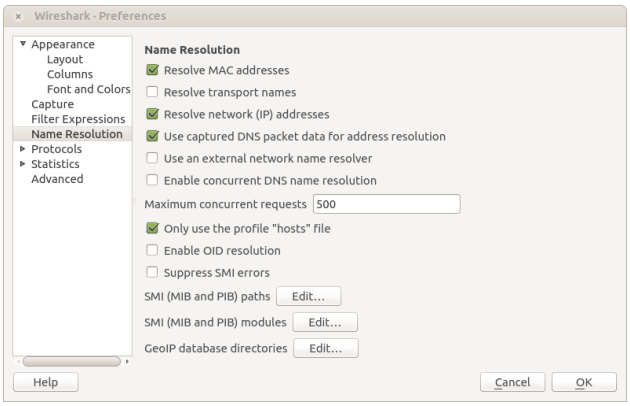



Figure 1.3: Activate a host file for name resolution in wireshark.

### Configuration of the DNS servers

It is necessary to configure the DNS service so SIP users can properly register in the proxy servers. To update the DNS server with some new Resource Records (RR), open a console in **router** (where the DNS service is located) and execute:

```
router$ nsupdate -k /etc/bind/rndc.key
> update add example.org. 172800   IN   NAPTR  30  0  "s"   "SIP+D2U"   ""  _sip._udp.example.org.
> update add example.org. 172800   IN   NAPTR  20  0  "s"   "SIP+D2T"   ""  _sip._tcp.example.org.
> update add _sip._udp.example.org. 3600 IN SRV 0 0 5060 prox1.example.org.
> update add _sip._tcp.example.org. 3600 IN SRV 0 0 5060 prox1.example.org.
> send
> update add example.com. 172800  IN   NAPTR    30 0 "s" "SIP+D2U" "" _sip._udp.example.com.
> update add example.com. 172800  IN   NAPTR    20 0 "s" "SIP+D2T" "" _sip._tcp.example.com.
> update add _sip._udp.example.com.   3600    IN      SRV     0 0 5060 prox2.example.com.
> update add _sip._tcp.example.com.   3600    IN      SRV     0 0 5060 prox2.example.com.
> send
```

To do this same configuration, you can directly use the label `dns`.

```
HOST$ simctl sip-2proxies exec dns
```

To test that all hosts are able to resolve the new resource records, you can do:

```
$ dig NAPTR example.org
$ dig SRV _sip._udp.example.org
$ dig prox1.example.org
$ dig NAPTR example.com
$ dig SRV _sip._udp.example.com
$ dig prox2.example.com
```

**Registration process**

In this exercise you are going to analyze the registration process of a user in the outbound proxy.

- Start `wireshark` in `SimNet0`.
- Start `pjsua` in **alice** with this command:

```
alice$ pjsua --id sip:alice@example.org  --registrar sip:example.org \
--realm example.org --username alice --password xxxx --null-audio --auto-play \
--play-file hello16000.wav --auto-answer 180 --outbound sip:example.org \
--add-codec=PCMA/8000 --nameserver=ns1.example.org
```

1. Have a look to each of these fields. What is the meaning of the `--id`? and `--registrar`, `--realm` and `--outbound`?
2. Now have a look to wireshark, and specially to the DNS dialogue. What is the purpose of the first DNS request sent by **alice**? What is the response to this request?
3. What is the purpose of the second DNS request sent by **alice**? What type of RR (Resource Record) is **alice** asking?
4. According to this request, what is the transport protocol for the SIP registration when using `pjsua`? and the domain of usage?
5. According to the response to the previous query, what is the SIP proxy for **alice**? Which RR is sending to inform **alice** about all this information? What is the TTL of this RR?
6. Is there any other additional RR in this response? Do these RR are of the same type of the previous one? and the TTL?

This is a typical DNS dialogue in which the client discovers the outbound proxy by means of the DNS system. In this case, the client software `pjsua` first asks for the DNS server in a first query (thanks to the option `--nameserver=ns1.example.org`). Once obtained the A record corresponding to this DNS server, `pjsua` performs a second query asking for a SRV record. As you can see, the current version of `pjsua` uses UDP as transport protocol for the registration process. This transport protocol may change in other versions of this softphone.

- Now let us continue with the SIP signaling.

    7. What new SIP method is **alice** using? What is the purpose of this method? What is the SIP URI of the request line?
    8. In this method, what are the contents of the `To` and `From` headers?
    9. Can you see any header indicating an expiry time for this request?
    10. What is the response for this first request? Is this response stating success or failure of the request?
    11. Does this response include any parameter in the header that should be used for the authentication of the user? Do you remember which is the formula that should be executed for this authentication?
    12. Is **alice** sending a second request? What is the difference of this second request regarding the first one?
    13. What is the response code for this second request? What does this mean?

    As you have seen, the REGISTER process when authentication is needed is not so simple. The user sends a REGISTER request to the SIP Registrar proxy. The `To` and `From` headers contain the AOR (address of record) of this user. The `Expires` header specifies the validity period (in seconds) of the registration. The proxy responds with a `401 Unauthorized`, which includes a `WWW-Authenticate` header containing a nonce that will be used to demonstrate that the user knows the corresponding password. After this, the user sends a second REGISTER containing an `Authorization` header, with the proof of the knowledge of the password. Remember that in the lecture slides you have the formula to verify the validity of these proofs. If the response is correct, the proxy sends a `200 Ok` response, meaning that the registration was successful.

    If you want to verify that **alice** is registered in the `kamailio` server, you can execute:

```
prox1$kamctl ul show
```

- Now, you are going to see the dialogue when the registration process fails. To make this, execute the following command in **bob**, having into account that it contains a wrong password:

```
bob$ pjsua --id sip:bob@example.com  --registrar sip:example.com \
--realm example.com --username bob --password yyyy --null-audio \
--auto-answer 180 --auto-loop --outbound sip:example.com \
--add-codec=PCMA/8000 --nameserver=ns1.example.com
```

    Have a look to the wireshark traces and locate the differences with the previous dialogue.

Before starting the next exercise, make the registering of **bob** in his proxy, and verify that it is correct.

```
bob$ pjsua --id sip:bob@example.com  --registrar sip:example.com \
--realm example.com --username bob --password xxxx --null-audio \
--auto-answer 180 --auto-loop --outbound sip:example.com \
--add-codec=PCMA/8000 --nameserver=ns1.example.com
```

### Call with Inbound and Outbound Proxies

Now that you understand the basics of DNS and SIP signaling, you are prepared to analyze a call using the classical sip trapezoid. Unlike the previous exercises in which we have guided you during the packet inspection by asking you specific questions, here you are completely free to analyze the traces on your own.

So, just make a call from **alice** to **bob**, and try to understand the overall dialogue, just considering these guidelines:

- As you will see, there will be many packets in both `SimNet0` and `SimNet1`, so we recommend to use the filter tool of `wireshark`.
- Making a picture with the packets involved may help you to understand the dialogue.

- Have a look to packets that are involved in the authentication of the clients.
- Pay attention to the most relevant SIP headers. This includes headers like `Via`, `Contact`, `Record-Route` and `Route`.